NANYANG TECHNOLOGICAL UNIVERSITY

EE6222 Machine Vision

Assignment 1: Dimensionality Reduction for Classification

MSc in Electrical and Electronic Engineering
Yan Ziming
G2507084J

October 29, 2025

Contents

1 Introduction

In this Assignment, I selected the MINIST dataset for training and testing the performance of classifiers. As for the dimension of images are largee, I explored several dimensionality reduction methods to reduce the dimension of data while trying to retain the most important features, including **PCA**, **LDA**, combined method (Fisherfaces), **LLE**, **Isomap**, and **PaCMAP**. I compared the effectiveness of these methods by plotting their performance with varying number of dimensions.

In addition, I establish several basic classifiers, like **MMD**, **KNN**, and include Logistic Regression Classifier with sklearn package. Therefore, I not only compare the effectiveness of different dimensionality reduction methods, but also access the accuracy of classifiy images for these classifer methods. I would describe the details of my experiment in the fllowing sections.

2 Dataset and Preprocessing

2.1 Dtaset Descrption

The MINIST dataset contains **70,000** images of handwritten digits (0-9) with a resolution of 28x28 pixels, which means there are **10** classes intotal. Each image had been saved with the format of vector with **784 in length**. In addition, the images are only have one channel, which means they are grayscale pixels. Because of the pre-shaped vectors, single channel, and easy to get features, I think MINIST dataset is suitable for this assignment. Therefore, I fetch the dataset from sklearn package directly 1.

Figure 1: Fetch Code

2.2 Normalization and Standarlization

Since the MINIST dataset had been well preprocessed and tested by plenty of researchers, I didn't do too much basic preprocessing, like outlier and missing value detection. However, I though normalization and standarlization arer still neccessary for the following basic classifiers, which may be affected intensely by the scle of features. So I rearrange the vectors to 0 mean and unit variance.

$$\bar{X} = \frac{X}{L-1}$$

$$X_{std} = \frac{X-\mu}{\sigma}$$

2.3 Dataset Partition

I found the number of samples (70,000) is too large, which may lead to many difficulties in my following experiment. Firstly, I will traing several classifiers with many dimension reduction methods, meaning the number of training process will be big. If I train 50,000-70,000 for each training, the consummed time will be very long. Secondly, my classifers which will be used are simple and bisic, therefore, they are easy to converge with small scale of data. With these two reason, I deciode to extract 7,000 samples from the origin dataset formly to reduce the expence of time.

After that, I split the dataset into training set and testing set through ***train_test_split*** function with ratio of 6:1 and finished the process of preparation of dataset 2.

```
Training Feature Dimension: (6000, 784)
Traing Label Dimension: (6000,)
Testing Feature Dimension: (1000, 784)
Testing Label Dimension: (1000,)
```

Figure 2: Dimension of practical dataset

3 Methodology

3.1 Linear Dimensionality Reduction

3.1.1 Linear Discriminant Analysis

Linear Discriminant Analysis (LDA) aims to find a linear combination of features with weight matrix **w** to transform the data into linear space, ensuring that the seperation between different classes is maximized while the variance with each class is minimized [1].

Step 1: Mean Value Computation

With given m classes $\{w_1, w_2, ... w_m\}$, there are c samples in each class w: $\{X_1, X_2, ... X_c\}$. For each sample vector, it has n features, $\{x_1, x_2, ... , x_n\}$. In this experiment, **n=784**.

Overall Mean:

$$\mu_t = \frac{1}{N}\sum_{i=1}^{N} x_i$$

Class Mean:

$$\mu_i = \frac{1}{n_i}\sum_{j=1}^{n_i} x_j$$

Where N is the total number of samples, and n_i is the number of samples in class w_i.

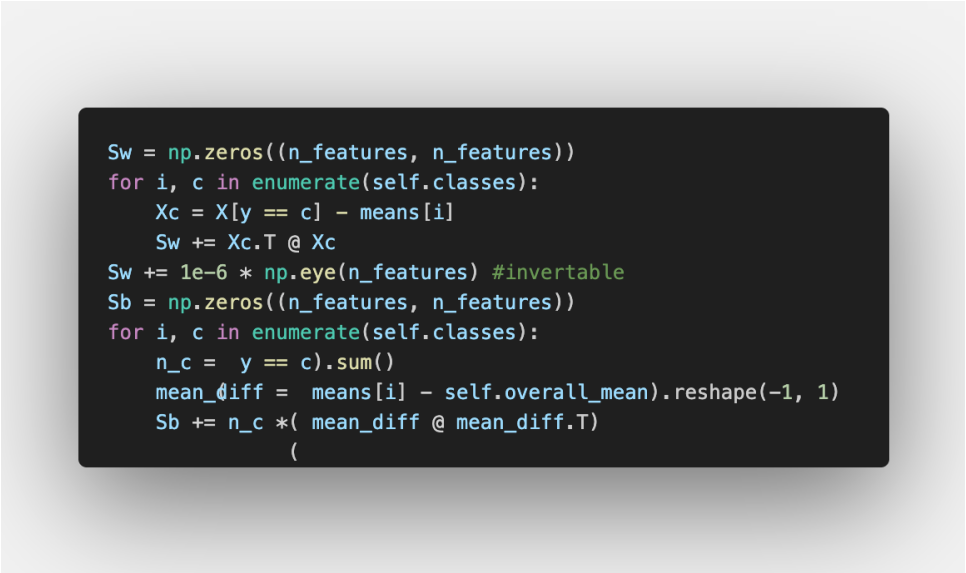
Step 2: Scatter Matrix Computation

Between-Class Scatter Matrix:

$$S_B = n_i * \sum_{j=1}^{m} (\mu_j - \mu_t)(\mu_j - \mu_t)^T$$

Within-Class Scatter Matrix:

$$S_W = \sum_{j=1}^{m} \sum_{x_i \in w_j} (x_i - \mu_j)(x_i - \mu_j)^T$$



```
Sw = np.zeros((n_features, n_features))
for i, c in enumerate(self.classes):
    Xc = X[y == c] - means[i]
    Sw += Xc.T @ Xc
Sw += 1e-6 * np.eye(n_features) #invertable
Sb = np.zeros((n_features, n_features))
for i, c in enumerate(self.classes):
    n_c =  y == c).sum()
    mean_diff =  means[i] - self.overall_mean).reshape(-1, 1)
    Sb += n_c *( mean_diff @ mean_diff.T)
              (
```

Figure 3: Scatter Computation Code

Step 3: Weight Matrix Computation

As I mentioned before, the LDA aims to find the optimal weight matrix to seperate classes on two dimension, equaling to the problem: maximize the ratio of between-class scatter to within-class scatter.

$$\mathbf{w}^* = \arg\max_w \frac{|w^T S_B w|}{|w^T S_W w|} \tag{1}$$

Since the $S_B w_i = \lambda_i S_W w_i$,

$$\lambda \mathbf{w} = S_W^{-1} S_B \mathbf{w} \tag{2}$$

Therefore, the $\mathbf{w}$ is the eigenvectors matrix according to eigenvalue λ, the largest λ refers to best matrix $\mathbf{w}^*$.

```
eigvals, eigvecs = eigh(Sb, Sw)
sorted_idx = np.argsort(eigvals)[::-1]
max_comp = min(len(self.classes) - 1, n_features)
n_comp = max_comp if self.n_components is None else min(self.
n_components, max_comp)
self.scalings = eigvecs[:, sorted_idx[:n_comp]]
```

Figure 4: Eigenvector Matrix Computation

Step 4: Transform Dataset

After getting the weight matrix, the transformation of input data $\mathbf{X}$ is:

$$\mathbf{Y} = \mathbf{X}\mathbf{w}^*$$

In practice, we will choose the k^{th} value of sorted eigenvalues to form $\mathbf{w}^*$. Theoritically, LDA is designed to project samples onto a lower-dimensional subspace, with the dimensionality being at most C - 1, because the rank of S_B is at most C-1, C is the number of classes.

3.1.2 Principal Component Analysis

Similar to LDA, Principal Component Analysis (PCA) is another effective method to dimensionality reduction. The difference is PCA can project the input data to different dimensions without limitaion like LDA. The projected data after PCA will also seperate largly, which means the redundency elements are eliminated [1].

Step 1: Data Processing

Before the projection, the data should be scaled and normalized, eliminating bias. Specifically, compute mean value to shift to 0 mean and covariance to investigate the distribution situation of data.

Covariance Matrix Computation:

$$\Sigma_i = \sum_{x_j \in w_i}^{n_i} (x_j - \mu_i)(x_j - \mu_i)^T$$

Where μ_i is the mean value of class w_i

Step 2: Eigenvectors Computation

The PCA aims to find the direction $\mathbf{w}$, the variance of data projected to that direction is largest. For the transformed data $Y = \mathbf{w}X$, variance computed:

$$Var(Y) = \frac{1}{N}\sum_{i=1}^{N}(w^T x_i)^2 = w^T \Sigma w$$

Solving the $\arg\max_w (w^T \Sigma w)$,

$$\Sigma \mathbf{w} = \lambda \mathbf{w} \tag{3}$$

Larger eigenvalue refers to better eigenvector, by sort the eigenvalues in descending sequence, we can fetch k^{th} large vlues and their correspondingly eigenvectors to form best matrix $\mathbf{w}^*$.

```
self.mean = np.mean(X, axis=0)
X_centered = X - self.mean
covariance_matrix = np.cov(X_centered, rowvar=False)
eigenvalues, eigenvectors = np.linalg.eigh(covariance_matrix)
sorted_indices = np.argsort(eigenvalues)[::-1]
self.components = eigenvectors[:, sorted_indices[:self.
n_components]]
```

Figure 5: PCA Realization Code

Step 3: Transform Dataset

With above process, we can get the corresponding optimal weight matrix, according to the input data and componenet coefficient k. Applying the matrix to input data, transformation finished.

3.1.3 Fisherfaces

Based on learning in course, I acknowledge that the reduction of feature equals to deleting redundant features, which can increase the accuray of models. However, when too much features deleted, the remaining features may not includes all the information we need, which causes the decrease of accuray backward. Therefore, the best accuarcy of LDA towards high dimension data is much lower than PCA normally. To overcome this situation, the Fisherfaces, which combines the LDA and PCA method is developed.

For realization, I just handle the data with PCA to roughly decrease the dimension firstly. Then, I applied LDA based on the corresponding PCA space to finish the whole reduction.

3.2 Nonlinear Dimensionality Reduction

A **manifold** is a topological space that locally resembles Euclidean space. Intuitively, even if data lie in a high-dimensional ambient space $\mathbb{R}^D$, their intrinsic degrees of freedom may be much smaller, forming a d-dimensional manifold ($d \ll D$). For example, images of a rotating object under fixed lighting conditions can be considered points on a 2D manifold embedded in a high-dimensional pixel space.

Formally, a dataset $X = \{x_1, x_2, \ldots, x_N\}$ is assumed to lie on or near a smooth manifold $\mathscr{M}$ of dimension d. The goal of **manifold learning** is to find a mapping $f : \mathbb{R}^D \to \mathbb{R}^d$ that preserves intrinsic geometric relationships among points.

Unlike linear methods such as PCA, which only capture global linear structures, manifold learning methods exploit the underlying nonlinear geometry. I include manifold learning methods for dimension reduction in this experiment.

3.2.1 Locally Linear Embedding

Brief Introduction:

Locally Linear Embedding (LLE) is a nonlinear dimensionality reduction algorithm that preserves local geometric relationships between neighboring data points. It assumes that each data point and its neighbors lie on or close to a locally linear manifold. LLE first reconstructs each data point as a linear combination of its nearest neighbors in the original high-dimensional space, then seeks a low-dimensional embedding that maintains these same reconstruction weights [2].

Let the dataset be $\{x_1, x_2, \ldots, x_N\}$, where each $x_i \in \mathbb{R}^D$. For each data point x_i, find its K nearest neighbors $\mathcal{N}(i)$ using Euclidean distance. Each point is reconstructed from its neighbors:

$$x_i \approx \sum_{j \in \mathcal{N}(i)} w_{ij} x_j$$

The reconstruction weights w_{ij} are obtained by minimizing the local reconstruction error:

$$\min_W \sum_{i=1}^{N} \left\| x_i - \sum_{j \in \mathcal{N}(i)} w_{ij} x_j \right\|^2 \tag{4}$$

subject to the constraint:

$$\sum_{j \in \mathcal{N}(i)} w_{ij} = 1$$

These weights capture the intrinsic geometry of the data.

In the embedding step, we find low-dimensional representations $y_i \in \mathbb{R}^d$ $(d \ll D)$ that preserve the same reconstruction weights:

$$\min_Y \sum_{i=1}^{N} \left\| y_i - \sum_{j \in \mathcal{N}(i)} w_{ij} y_j \right\|^2 \tag{5}$$

subject to:

$$\sum_i y_i = 0, \quad \frac{1}{N} \sum_i y_i y_i^T = I$$

The optimal embedding $Y = [y_1, \ldots, y_N]^T$ is obtained by computing the bottom $(d+1)$ eigenvectors of the matrix:

$$M = (I - W)^T (I - W) \tag{6}$$

and discarding the smallest eigenvector corresponding to the zero eigenvalue.

Implementation:

In this experiment, I implemted the LLE by using the `LocallyLinearEmbedding` class from `scikit-learn`. The key parameters are the number of neighbors (`n_neighbors`) and the target embedding dimension (`n_components`). Below shows an example code used in this experiment:

```
reduction_m = LocallyLinearEmbedding(n_components=n)
reduction_m.fit(X_train)
X_train_reduced = reduction_m.transform(X_train)
X_test_reduced = reduction_m.transform(X_test)
```

Figure 6: LLE Realization Code

3.2.2 Isometric Mapping

Brief Introduction:

Isometric Mapping (Isomap) is a nonlinear dimensionality reduction method based on the manifold learning assumption, which posits that high-dimensional data lie on a low-dimensional nonlinear manifold. Unlike linear methods such as PCA, Isomap seeks to preserve the intrinsic geometry of the manifold by approximating geodesic distances between points [3].

For a dataset $\{x_1, x_2, \ldots, x_N\} \subset \mathbb{R}^D$, Isomap constructs a neighborhood graph connecting each point to its K nearest neighbors. The edges are weighted by the Euclidean distance between connected points. The geodesic distance between any two points x_i and x_j is then approximated by the shortest path distance in this graph:

$$D_G(i,j) = \min_{\text{paths } i \to j} \sum_{(p,q) \in \text{path}} \|x_p - x_q\| \tag{7}$$

Once the matrix of geodesic distances D_G is obtained, classical Multidimensional Scaling (MDS) is applied to compute a low-dimensional embedding $Y \in \mathbb{R}^{N \times d}$ that best preserves these distances. The centered inner-product matrix is calculated as

$$B = -\frac{1}{2} H D_G^2 H, \quad \text{where } H = I - \frac{1}{N}\mathbf{1}\mathbf{1}^T \tag{8}$$

and the embedding is obtained from the top d eigenvectors of B:

$$Y = V_d \Lambda_d^{1/2}$$

where Λ_d contains the d largest eigenvalues, and V_d the corresponding eigenvectors. The resulting low-dimensional coordinates preserve the manifold's intrinsic global geometry.

Implementation:

Isomap can be implemented using the `Isomap` class from `scikit-learn`. The main parameters are the number of neighbors (`n_neighbors`) and the target embedding dimension (`n_components`). The code snippet used in this work is shown in Figure 7, and the resulting 2D embedding is visualized for comparison with other methods.

```
reduction_m = Isomap(n_components=n)
reduction_m.fit(X_train)
X_train_reduced = reduction_m.transform(X_train)
X_test_reduced = reduction_m.transform(X_test)
```

Figure 7: Isomap Realization Code

3.2.3 Pairwise Controlled Manifold Approximation

Brief Introduction:

PaCMAP is a modern nonlinear dimensionality reduction method designed to preserve both local and global structures in high-dimensional data. Unlike traditional methods such as t-SNE and UMAP, which primarily focus on local structure, or TriMAP, which emphasizes global relationships, PaCMAP balances these by controlling three types of point-pair relationships:

- **Neighbor pairs:** Preserve local neighborhood structure.
- **Mid-near pairs:** Preserve medium-range relationships.
- **Far pairs:** Prevent collapse of distant points and maintain global separation.

The main objective of PaCMAP is to optimize a low-dimensional embedding such that these point-pair relationships are faithfully preserved, producing a high-quality representation suitable for visualization and downstream tasks [4]. PaCMAP's objective function consists of three components corresponding to the three types of point pairs:

$$L_{\text{PaCMAP}} = L_{\text{neighbors}} + L_{\text{mid-near}} + L_{\text{far}} \tag{9}$$

where:

- $L_{\text{neighbors}}$ measures the discrepancy of neighbor pairs in low-dimensional space relative to the original space.
- $L_{\text{mid-near}}$ measures the preservation of mid-range distances.
- L_{far} ensures that distant points remain well separated.

The specific loss formulations and weighting schemes can be found in the original paper [4].

Implementation: As mentioned in to essay, developers have embeded PaCMAP in Python. We can implement the reduction method by using the `pacmap` package as follows:

```
embedding = pacmap.PaCMAP(
    n_components=n,
    n_neighbors=10,
    MN_ratio=0.5,
    FP_ratio=2.0
)
X_reduced=embedding.fit_transform(X)
```

Figure 8: PaCMAP Realization Code

Since it is a very new tool designed to reduce the dimension of data to visualiza the relation between high dimension features, `pacmap` only allows to reduce the total dataset, different from the previous methods package usage, which can be seen by comparing the realization of PaCMAP (Figure 8) with Isomap (Figure 7) or LLE (Figure 6).

3.3 Classifiers Establishment

After reducing the dimension, I need to verify the influence of these reduction methods. Therefore, I need to develope classifiers and use their performance (accuracy) to judge the effection of datashape changes.

3.3.1 Minimun Mahalanobis Distance Classifier

Mahalanobis Distance represents the difference between two images. The MMD classifier refers to divide the input image data to the class which contains the nearest image data.

The Mahalanobis Distance of sample $X = \{x_1, x_2, ...\}$ to class w_j:

$$D_j = (X - \mu_j)\Sigma_j^{-1}(X - \mu_j)^T$$

Where μ_j is the mean value of data in class, Σ_j is the covariance of class data.

The above propcess is the fitting part of classifier, which means the model will try to find appropriate parameters through the training data. I implement it with codes in Figure 9. By the way, there is possible that covariance may be singular, leading no inversion. So I added the covariance with `1e-6 * np.eye(X.shape[1])` to avoid singular condition.

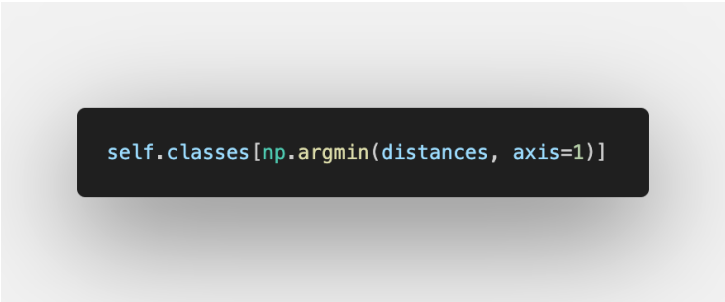
```python
def fit(self, X, y):
        self.classes = np.unique(y)
        self.covariance = np.cov(X, rowvar=False) + 1e-6 * np.eye(X.shape[1])
        self.inv_covariance = np.linalg.inv(self.covariance)
        self.means = []
        for w in self.classes:
            X_w = X[y == w]
            mean_w = np.mean(X_w, axis=0)
            self.means.append(mean_w)
        self.means = np.array(self.means)
```

Figure 9: Fitting of MMD Classifier

After getting the distance of input point to each classes, the predicted label of input point is considered to

$$\arg\min_{w_j}(D(X, w_j))$$

I successfully simulated this selecting processs with `np.argmin()` function enpackaged in predict function.



```python
self.classes[np.argmin(distances, axis=1)]
```

Figure 10: Optimal Class Selection Code

3.3.2 K-Nearest Neighbors Classifier

Basically, the foundamental principle of MMD and KNN follows are same: just consider the sample with smaller distance as the more similar points. If one class has plenty of similar points, the test sample have higher probability belongs to corresponding class. However, their actual realization methods are different.

The MMD pay attention to the class, use mean in calss to represente the class feature and find the similarity between class mean and test sample. The KNN concentrate on the samples individually. The mechanism is to find the distances of test sample with each points in training, select the first k nearest neighbors, and assigns the test point with the label appears most frequently among k points, which is called the Voting process Figure 11.

Generallly, the realization process is similar to the MMD except the method of assigning label prediction.

Figure 11: Voting Process of KNN

3.3.3 Logistic Regression Classifier

The logistic regression classifier models the probability that the output label y equals 1 given an input feature vector $x \in \mathbb{R}^d$:

$$h_\theta(x) = P(y = 1 \mid x)$$

To map the linear function $\theta^T x$ into the interval $[0,1]$, we use the **sigmoid function**:

$$h_\theta(x) = \sigma(\theta^T x) = \frac{1}{1+e^{-\theta^T x}}$$

where

$$\sigma(z) = \frac{1}{1+e^{-z}}.$$

Classification is made according to:

$$\hat{y} = \begin{cases} 1, & \text{if } h_\theta(x) \geq 0.5, \\ 0, & \text{if } h_\theta(x) < 0.5. \end{cases}$$

Equivalently, the **decision boundary** satisfies:

$$\theta^T x = 0.$$

The cost function (negative log-likelihood) is defined as:

$$J(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \left[y_i \log(h_\theta(x_i)) + (1-y_i)\log(1-h_\theta(x_i)) \right].$$

It is derived from the likelihood:

$$L(\theta) = \prod_{i=1}^{N} P(y_i \mid x_i; \theta)$$

and the corresponding log-likelihood:

$$\log L(\theta) = \sum_{i=1}^{N} \left[y_i \log(h_\theta(x_i)) + (1-y_i)\log(1-h_\theta(x_i)) \right].$$

The gradient of the cost function is given by:

$$\frac{\partial J(\theta)}{\partial \theta} = \frac{1}{N}\sum_{i=1}^{N} (h_\theta(x_i) - y_i) x_i.$$

The parameters are updated iteratively using:

$$\theta := \theta - \alpha \frac{\partial J(\theta)}{\partial \theta},$$

where α is the learning rate.

The implementation of Logistic Regression Classifier is easy, because it had been included in `sklearn` package, I can establish it with simple codes:

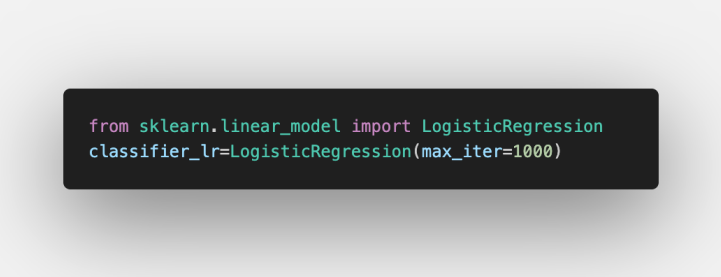


Figure 12: Implentation of Logistic Regression Classifier

This classifier has much more complex mechanism compared with KNN and MMD, therefore, this is the only one model I didn't write its program in myself.

3.4 Evaluation Methods

To evaluate the performance of different dimensionality reduction techniques, we used classification accuracy as the main metric. After embedding the high-dimensional data into lower-dimensional space (using PCA, LDA, LLE, Isomap, and PaCMAP), the reduced features were fed into different classifiers for supervised evaluation. I compared the classification accuracy across different reduced dimensions (e.g., 2D, 5D, 10D, ...) to analyze how each method preserves discriminative information.

The experimental results and visual comparisons will be discussed in the following *Results and Discussion* section.

4 Results and Discussion

Since the too huge dimension of origin data (784 features) and time costs of training, I use reduction methods to reduce the data to discreted dimension parameter n, which I selected as $\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 50, 100, 300, 700\}$. To show their differences, I plotted the accuracys corresponding to various reduced test data.

I divided the output graphs into two parts: performance of classifiers with linear reduction method and manifold reduction method to avoid the mess of too much lines in one graph. As for the baseline for comparation, I used the accuarcy of corresponding classifier trained with original data as baseline.

As I said, I selected several target dimensions to decrease the loop number. Their graphs with original x-axis is difficult for reading with different intervals. Therefore, I applied the log method on the x-axis of graphs to simplify.

4.1 Linear Reduction Method Performance

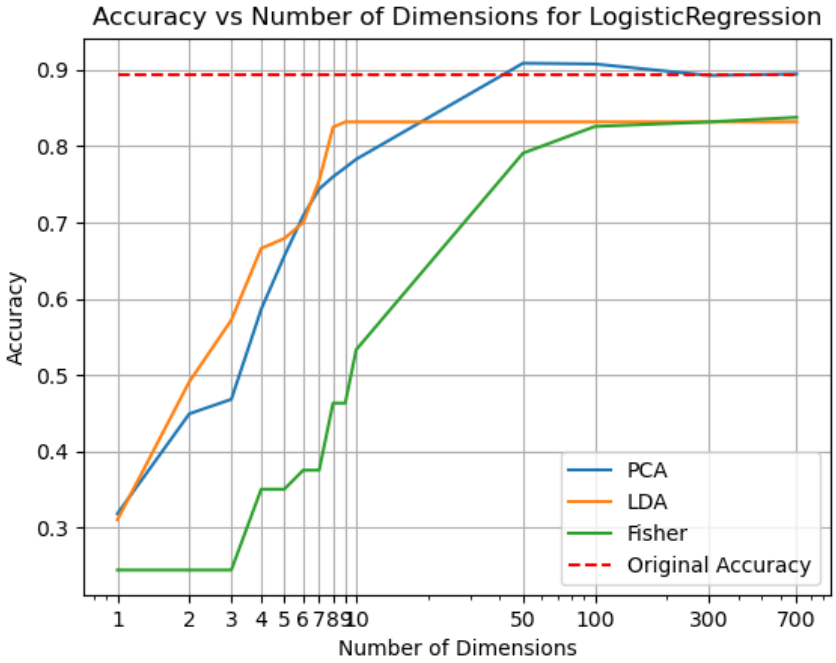


Figure 13: Linear Reduction Performance with Linear_regression

This is the performance graph of Linear Regression Classifier, I applied PCA, LDA, and Fisherfaces this three methods for reduction.

Generally, with the increasing of reduced dimension, the accuracys of each method are increasing as well. This phenomenon is more apparent when the dimension is less than 10. The increasing feature add information that could be used to predict the test label, therefore, the accuracy increases intensely. When the dimension becomes bigger, the accuracy increasing rate slow down.

Individually, the LDA method accuracy remains constant after the 10th dimension, which follows the theory that LDA could only reduce the data to C-1 dimensions, where C is the number of features (full dimension). As for the PCA, when the dimension increased to about full dimension, I find that the accuracy decreased a bit compared with middle points, not increasing as before. According to the previous introduction in Fisher, the too much features will infuence the classifier judge with trash information containing in some features, therefore, the accuracy may decrease a bit with high dimension. This is the reason of Fisherfaces developed. The Fisher indeed remains constant for high dimensions.

By comparison, the PCA method could increase the accuracy to the baseline, even better with appropriate reduced dimension. The final LDA is much lower than baseline, with most features are neglected. As for Fisher, it combines the PCA and LDA, which overcomes the limitation of dimension, but still delete many features, leading to lower accuracy compared with PCA and baseline. This outcomes shows that the Fisher method may not suitable for this written number dataset.

4.2 Manifold Reduction Method Performance

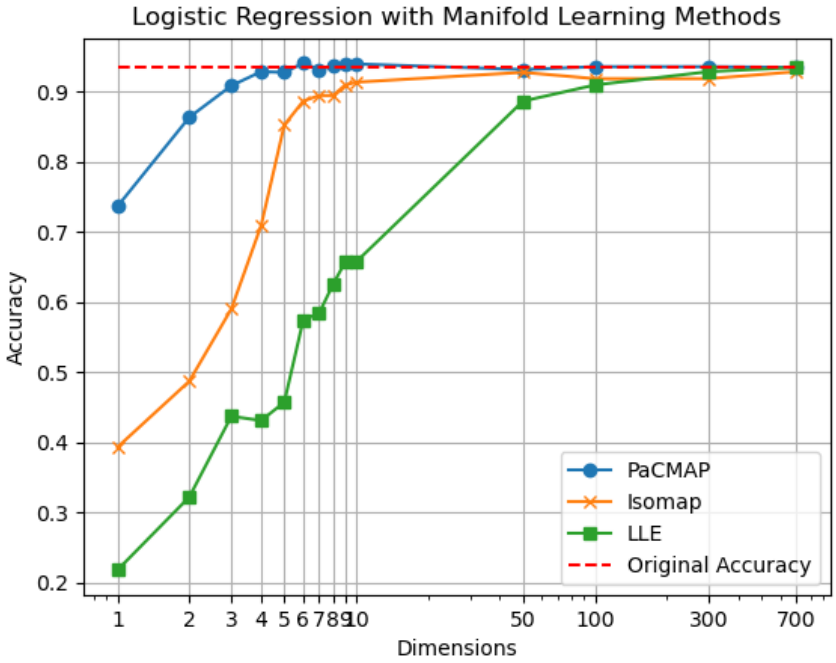


Figure 14: Manifold Reduction Performance with Linear_regression

The above figure is the performance of three manifold reduction method: LLE, Isomap, and PaCMAP. Similar to linear method, with higher the dimension used, the better prediction accuracy will be generally.

By comparing the linear method performance and manifold performance, I have following findings:

1. Among all methods, the PaCMAP method seems have fast convergence speed. When the dimension increase to about 4 or 5, the accuracy score doesn't change anymore. In other words, PaCMAP could represented the principle feature of MINIST data most efficiently. PaCMAP is the most sauitable method for reducing dimension for this dataset.

2. For MINIST dataset, if we want to use low dimension for machine learning, Manifold may be more effective, because the accuracys of Isomap and PaCMAP are much higher than PCA and LDA for dimension in interval [5,10]. This may suggest that the MNIST dataset exhibit strong non-linear structures in its feature space.

4.3 Other Classifiers

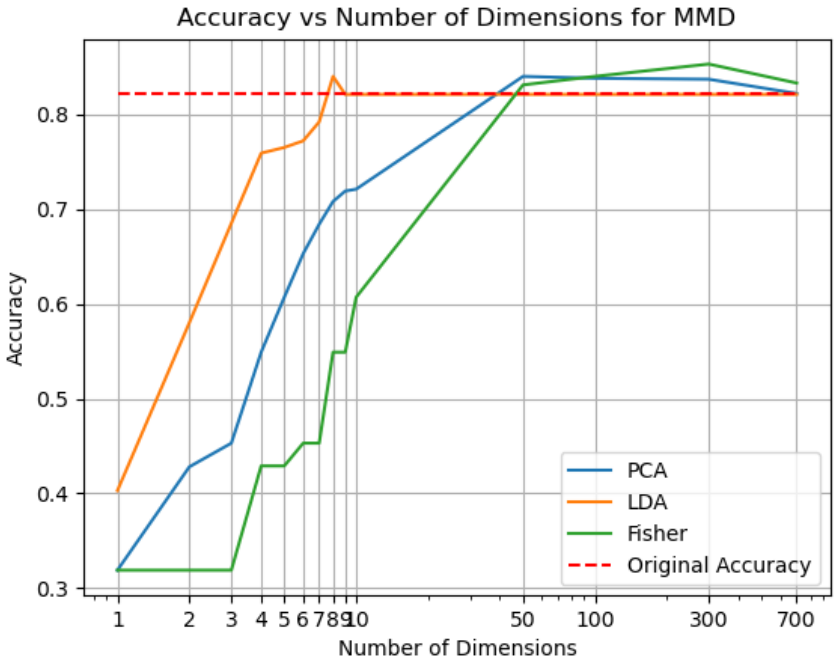


Figure 15: Linear Reduction Performance with MMD

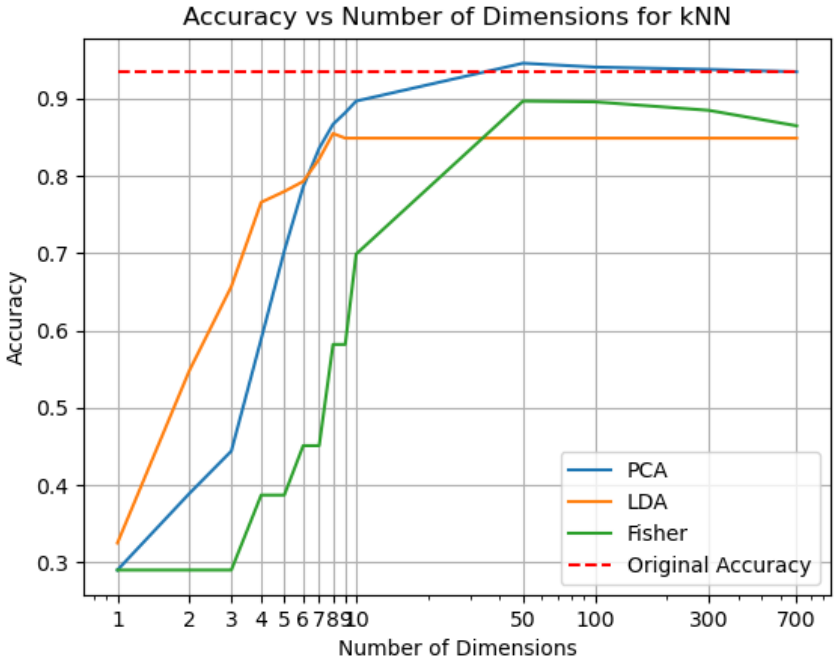


Figure 16: Linear Reduction Performance with KNN

In this experiment, I applied other classifiers in addition to the Linear Regression. Since the manifold methods are more complex than linaer methods, I only applied linear methods for

three classifiers to find most appropriate classifier for testing by comparing accuracys and data feature. Finally, I decide to use Linear Regression for follwing nonlinear methods. However, I still want to show the performances of MMD and KNN here (Figure 15, 16).

5 Conclusion

In this experiment, I used 6 reduction method and compared their performance on logistic regression classifier, based on the MINIST dataset. In addition, I compared the performance of different classifiers with same situation.

The most effective reduction method in this experiment is PaCMAP, which can reduce the 784 feature data to less than 10 dimensions with good performance accuracy. By the way, the PaCMAP is designed to show the relationship of data in 2 dimension, so its effectiveness hasn't been proved on higher dimension. According to my experiment, it is effective for MINIST dataset for dimensions other than 2 at least, which can be considered as one rough proof for its effectiveness on other dimensions basically.

References

[1] I. K. Fodor, "A Survey of Dimension Reduction Techniques," Tech. Rep. UCRL-ID-148494, 15002155, May 2002.

[2] S. T. Roweis and L. K. Saul, "Nonlinear dimensionality reduction by locally linear embedding," *Science*, vol. 290, no. 5500, pp. 2323–2326, 2000.

[3] J. B. Tenenbaum, V. de Silva, and J. C. Langford, "A global geometric framework for nonlinear dimensionality reduction," *Science*, vol. 290, no. 5500, pp. 2319–2323, 2000.

[4] Y. Wang, Y. Sun, H. Huang, and C. Rudin, "Dimension Reduction with Locally Adjusted Graphs," Dec. 2024. arXiv:2412.15426 [cs].

Appendix

```
## import libs
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
# from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression

##include minist dataset
mnist = fetch_openml('mnist_784', version=1)
print(mnist.keys())
X_t, y_t = mnist["data"], mnist["target"]
## decrese the size of dataset for faster computation
X,_,y,_ =train_test_split(X_t, y_t, train_size=7000, stratify=y_t, random_state=0)

print(X.shape)
print(y.shape)

##Preprocessing
#alignment
X = X.to_numpy()
X = X.astype(np.float32)
y = y.astype(np.int64)
y = y.to_numpy()
#normalization
X = X / 255.0
# print(X[0])
# print(y[0])

##Split the data set into training set and test set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=1/7,
random_state=0)
print("Training Feature Dimension:",X_train.shape)
print("Traing Label Dimension:",y_train.shape)
print("Testing Feature Dimension:",X_test.shape)
print("Testing Label Dimension:",y_test.shape)
```

```python
## Mnimun Mahalanobis Distance Classifier
class MMD:
    def __init__(self):
        pass

    def fit(self, X, y):
        self.classes = np.unique(y)
        self.covariance = np.cov(X, rowvar=False) + 1e-6 * np.eye(X.shape[1])
        self.inv_covariance = np.linalg.inv(self.covariance)
        self.means = []
        for w in self.classes:
            X_w = X[y == w]
            mean_w = np.mean(X_w, axis=0)
            self.means.append(mean_w)
        self.means = np.array(self.means)

    def predict(self, X):
        diff = X[:, np.newaxis] - self.means
        distances = np.einsum('ijk,kl,ijl->ij', diff, self.inv_covariance, diff)
        return self.classes[np.argmin(distances, axis=1)]

    def score(self, X, y):
        y_pred = self.predict(X)
        return np.mean(y_pred == y)
```

```python
## kNN Classifier
class kNN():
    def __init__(self,k):
        self.k=k

    def fit(self,X,y):
        self.classes=np.unique(y)
        self.data=X
        self.d_label=y


    def Distance(self,d,t):
        return np.sum((d-t)*(d-t))

    def predict(self,X_test):
        y_pred=[]
        for x in X_test:
            dists=[]
            for data in self.data:
                dists.append(self.Distance(data,x))
            dists=np.array(dists)
            idk=np.argpartition(dists,self.k)[0:self.k]
            klabels=self.d_label[idk]
            labels, counts=np.unique(klabels,return_counts=True)
            y_pred.append(labels[np.argmax(counts)])
        y_pred=np.array(y_pred)
        return y_pred

    def score(self,X_test,y_test):
        y_pred=self.predict(X_test)
        accuracy=np.mean(y_pred == y_test)
        return accuracy
```

```python
classifier_mmd=MMD()
classifier_mmd.fit(X_train, y_train)
y_pred_mmd=classifier_mmd.predict(X_test)
accuracy_mmd=classifier_mmd.score(X_test,y_test)
print("MMD accuracy:", accuracy_mmd)

classifier_knn=kNN(k=3)
classifier_knn.fit(X_train,y_train)
y_pred_knn=classifier_knn.predict(X_test)
accuracy_knn=classifier_knn.score(X_test,y_test)
print("kNN accuracy:", accuracy_knn)

from sklearn.linear_model import LogisticRegression
classifier_lr=LogisticRegression(max_iter=1000)
classifier_lr.fit(X_train, y_train)
y_pred_lr=classifier_lr.predict(X_test)
accuracy_lr=classifier_lr.score(X_test,y_test)
print("Logistic Regression accuracy:", accuracy_lr)
```

```python
## PCA for Dimension Reduction
class PCA:
    def __init__(self, n_components):
        self.n_components = n_components
        self.components = None
        self.mean = None

    def fit(self, X):
        self.mean = np.mean(X, axis=0)
        X_centered = X - self.mean
        covariance_matrix = np.cov(X_centered, rowvar=False)
        eigenvalues, eigenvectors = np.linalg.eigh(covariance_matrix)
        sorted_indices = np.argsort(eigenvalues)[::-1]
        self.components = eigenvectors[:, sorted_indices[:self.n_components]]

    def transform(self, X):
        X_centered = X - self.mean
        return np.dot(X_centered, self.components)
```

```
## LDA for Dimension Reduction
from scipy.linalg import eigh

class LDA:
    def __init__(self, n_components=None):
        self.n_components = n_components
        self.scalings = None
        self.overall_mean = None
        self.classes = None

    def fit(self, X, y):
        X = np.asarray(X, dtype=float)
        y = np.asarray(y)
        n_samples, n_features = X.shape
        self.classes = np.unique(y)
        self.overall_mean = X.mean(axis=0)
        means = np.array([X[y == c].mean(axis=0) for c in self.classes])
        Sw = np.zeros((n_features, n_features))
        for i, c in enumerate(self.classes):
            Xc = X[y == c] - means[i]
            Sw += Xc.T @ Xc
        Sw += 1e-6 * np.eye(n_features) #invertable
        Sb = np.zeros((n_features, n_features))
        for i, c in enumerate(self.classes):
            n_c =  y == c).sum()
            mean_diff =  means[i] - self.overall_mean).reshape(-1, 1)
            Sb += n_c *( mean_diff @ mean_diff.T)
        eigvals, eigvec$ = eigh(Sb, Sw)
        sorted_idx = np.argsort(eigvals)[::-1]
        max_comp = min(len(self.classes) - 1, n_features)
        n_comp = max_comp if self.n_components is None else min(self.n_components,
max_comp)
        self.scalings = eigvecs[:, sorted_idx[:n_comp]]
        return self

    def transform(self, X):
        if self.scalings is None:
            raise RuntimeError("Call fit first.")
        X = np.asarray(X, dtype=float)
        Xc = X - self.overall_mean
        return Xc @ self.scalings
```

```python
#import lda
def model_accuracy(dimensions, X_train, y_train, X_test, y_test, model, method):
    accuracys=[]
    for dim in dimensions:
        # reduction method selection
        if method == 'PCA':
            reduction_m = PCA(n_components=dim)
            reduction_m.fit(X_train)
            X_train_reduced = reduction_m.transform(X_train)
            X_test_reduced = reduction_m.transform(X_test)
        elif method == 'LDA':
            reduction_m = LDA(n_components=min(dim, len(np.unique(y_train))-1))
            reduction_m.fit(X_train,y_train)
            X_train_reduced = reduction_m.transform(X_train)
            X_test_reduced = reduction_m.transform(X_test)
        elif method == 'Fisher'
            pca_dim = max(dim // 2, 1)
            lda_dim = max(dim - pca_dim, 1)
            pca = PCA(n_components=pca_dim)
            pca.fit(X_train)
            X_train_pca = pca.transform(X_train)
            X_test_pca = pca.transform(X_test)

            lda = LDA(n_components=min(lda_dim, len(np.unique(y_train)) - 1))
            lda.fit(X_train_pca, y_train)
            X_train_reduced = lda.transform(X_train_pca)
            X_test_reduced = lda.transform(X_test_pca)
        else:
            raise ValueError("Unknown method")
        # classification model selection
        if model == 'MED':
            classifier = MED()
        elif model == 'MMD':
            classifier = MMD()
        elif model == 'kNN':
            classifier = kNN(3)
        elif model == 'LogisticRegression':
            classifier = LogisticRegression(max_iter=1000)
        else:
            raise ValueError("Unknown model")
        # evaluation
        classifier.fit(X_train_reduced, y_train)
        accuracy = classifier.score(X_test_reduced, y_test)
        accuracys.append(accuracy)
    return np.array(accuracys)

def arrangement(dimensions, X_train, y_train, X_test, y_test, models, methods):
    accuracys_all=[]
    refer_acc=[]
    for model in models:
        accuracys_model=[]
        if model == 'MED':
            classifier = MED()
        elif model == 'MMD':
            classifier = MMD()
        elif model == 'kNN':
            classifier = kNN(3)
        elif model == 'LogisticRegression':
            classifier = LogisticRegression(max_iter=1000)
        else:
            raise ValueError("Unknown model")
        classifier.fit(X_train, y_train)
        accuracy_origin = classifier.score(X_test, y_test)
        refer_acc.append(accuracy_origin)
        for method in methods:
            accuracys_method=model_accuracy(dimensions, X_train, y_train, X_test,
y_test, model, method)
            accuracys_model.append(accuracys_method)
        accuracys_all.append(accuracys_model)
    return np.array(accuracys_all), np.array(refer_acc)
```

```
def visualize(dimensions,accuracys,models,methods,refer_acc):
    for i, model in enumerate(models):
        x=dimensions
        y=accuracys[i]
        plt.figure()
        for j, method in enumerate(methods):
            plt.plot(x,y[j], label=method)
        origin_acc=np.zeros(x.shape)+refer_acc[i]
        plt.plot(x,origin_acc, label='Original Accuracy', linestyle='--', color=
'red')
        plt.title(f'Accuracy vs Number of Dimensions for {model}')
        plt.xscale('log')
        plt.xticks(dimensions, dimensions)
        plt.xlabel('Number of Dimensions')
        plt.ylabel('Accuracy')
        plt.legend()
        plt.grid()
        plt.savefig(f'Report/Images/{model}_accuracy.png')
        plt.show()

dimensions = np.array([1,2,3,4,5,6,7,8,9,10,50,100,300,700])
models=[ 'LogisticRegression','MMD','kNN']
methods =  'PCA', 'LDA', 'Fisher']
accuracys_total, refer_acc=arrangement(dimensions, X_train, y_train, X_test, y_test
, models, methods)
# print(accuracys_total)
visualize(dimensions,accuracys_total,models,methods,refer_acc)
```

```
## Isomap for Dimension Reduction
from sklearn.manifold import Isomap
dimensions=np.array([1,2,3,4,5,6,7,8,9,10,50,100,300,700])
accuracys_isomap=[]
for n in dimensions:
    reduction_m = Isomap(n_components=n)
    reduction_m.fit(X_train)
    X_train_reduced = reduction_m.transform(X_train)
    X_test_reduced = reduction_m.transform(X_test)
    classifier_lr=LogisticRegression(max_iter=1000)
    classifier_lr.fit(X_train_reduced, y_train)
    y_pred_lr=classifier_lr.predict(X_test_reduced)
    accuracy_lr=classifier_lr.score(X_test_reduced,y_test)
    accuracys_isomap.append(accuracy_lr)
```

```
## LLE for Dimension Reduction
from sklearn.manifold import LocallyLinearEmbedding
accuracys_lle=[]
for n in dimensions:
    reduction_m = LocallyLinearEmbedding(n_components=n)
    reduction_m.fit(X_train)
    X_train_reduced = reduction_m.transform(X_train)
    X_test_reduced = reduction_m.transform(X_test)
    classifier_lr=LogisticRegression(max_iter=1000)
    classifier_lr.fit(X_train_reduced, y_train)
    y_pred_lr=classifier_lr.predict(X_test_reduced)
    accuracy_lr=classifier_lr.score(X_test_reduced,y_test)
    accuracys_lle.append(accuracy_lr)
```

```
## use pacmap for Dimension Reduction
import pacmap
accracys=[]
for n in dimensions:
    embedding = pacmap.PaCMAP(
        n_components=n,
        n_neighbors=10,
        MN_ratio=0.5,
        FP_ratio=2.0
    )
    X_reduced=embedding.fit_transform(X)
    X_train_reduced, X_test_reduced, y_train, y_test = train_test_split(X_reduced,
y, test_size=1/7, random_state=0)
    clf = LogisticRegression(max_iter=1000)
    clf.fit(X_train_reduced, y_train)
    accuracy = clf.score(X_test_reduced, y_test)
    accracys.append(accuracy)
accuracys=np.array(accracys)
```

```
plt.figure()
plt.plot(dimensions, accuracys, label='PaCMAP', marker='o')
plt.plot(dimensions, accuracys_isomap, label='Isomap', marker='x')
plt.plot(dimensions, accuracys_lle, label='LLE', marker='s')
plt.plot(dimensions, np.zeros(dimensions.shape)+accuracy_lr, label=
'Original Accuracy', linestyle='--', color='red')
plt.xscale('log')
plt.xticks(dimensions, dimensions)
plt.legend()
plt.grid()
plt.title('Logistic Regression with Manifold Learning Methods')
plt.xlabel('Dimensions')
plt.ylabel('Accuracy')
##save figure to Report/Images
plt.savefig('Report/Images/manifold_methods_accuracy.png')
plt.show()
```